CS683: Advanced Computer Architecture

Programming Assignment 1 — Task 2

Hardware-Conscious Matrix Multiplication for Inference

Group Members:
Kaarmukilan B (23B1215)
Nandana MA (23B1216)
Maitranya Patil (23B1239)
Sreesti Sahoo (23B1267)

September 7, 2026

1. Objective

Task 2 studies the optimization of single-precision general matrix multiplication (SGEMM), a central kernel in neural-network and LLM inference. The objective is to improve the performance of the provided baseline by exploiting architectural features of a modern x86 processor, specifically SIMD execution, cache blocking, software prefetching, and their combination.

The matrix multiplication uses the "NT" layout

$$C[i, j] = \sum_{p=0}^{K-1} A[i, p]\, B[j, p],$$

where A is $M \times K$, B is stored as $N \times K$, and C is $M \times N$. Therefore, the contraction dimension K is contiguous for both input matrices. The operation performs

$$2MNK$$

floating-point operations.

The assignment requires analysis of:

- software prefetching and cache-fill level,
- SIMD width and instruction count,
- the combined optimized kernel,
- performance across different matrix sizes,
- cache-miss behavior, and
- the effect of hardware prefetchers.

2. Experimental Setup

Table 1: Experimental platform

Processor	13th Gen Intel(R) Core(TM) i5-13420H
Architecture	Intel x86-64
L1-D cache	40 KiB per core (320 KiB total across 8 cores)
L2 cache	7 MiB total (5 instances)
LLC / L3 cache	12 MiB (1 instance)
Compiler	gcc/g++ (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0
OS	Ubuntu 24.04.4 LTS
Hardware prefetchers	Disabled for software-prefetch experiments (Task 2A) enabled for SIMD and optimized (combined) experiments (Task 2B,2C).

3. Baseline SGEMM

The provided reference implementation computes each output element as an independent dot product. Because $A[i, p]$ and $B[j, p]$ are contiguous along the K-dimension, the innermost loop naturally has unit-stride accesses. However, the scalar baseline does not exploit SIMD lanes, register-level reuse, or explicit prefetching.

4. TASK 2A: Software Prefetching

4.1. Purpose of Software Prefetching

Software prefetching attempts to overlap memory latency with useful computation. The prefetch distance must be large enough that data arrives before demand use, but not so large that prefetched lines are evicted before use or pollute the cache.

4.2. Prefetch Distance Study

Table 2: Prefetch-distance sweep

Distance (floats)	Distance (cache lines)	Time (s)	Speedup	Notes
16	1	0.876	0.63×	
32	2	0.884	0.63×	
64	4	0.946	0.59×	
128	8	1.037	0.54×	

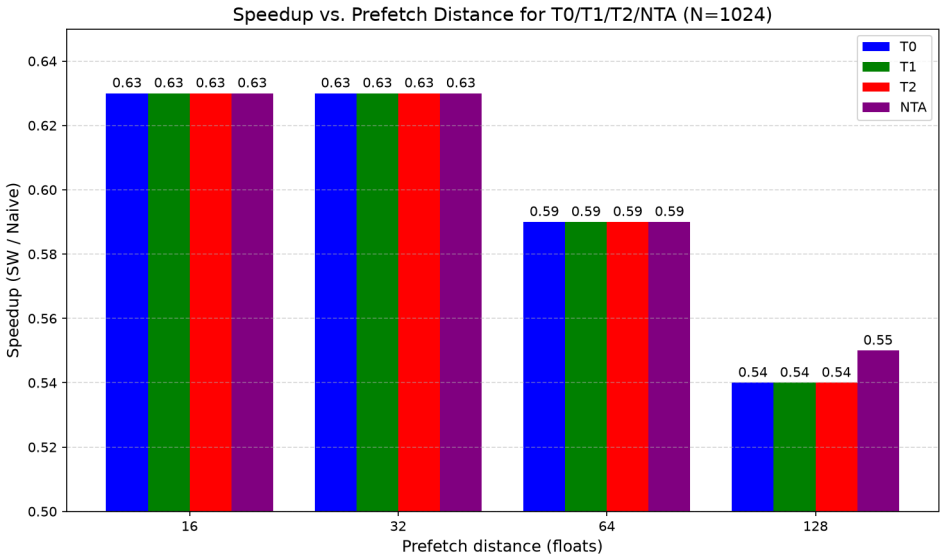

Figure 1: Speedup vs. prefetch distance

Observation. The best measured prefetch distance is 16 floats (1 cache line) for N=1024, yielding a speedup of 0.63× relative to naive. The trend is that increasing the prefetch distance monotonically degrades performance: 32→0.63×, 64→0.59×, and 128→0.54×. At small distances (e.g., 16), prefetch requests arrive just in time, keeping the working set close to the core. At excessive distances (128 floats), the prefetches are premature; they evict useful L1/L2 lines before they are needed, causing cache pollution. This is clearly supported by the T0 cache-miss data for N=1024: L1 misses rise from 3.0M (d=16) to 25.7M (d=128), while LLC misses increase from 34k to 659k over the same range, confirming that overly distant prefetches contaminate the cache hierarchy.

4.3. Cache Fill Level / Prefetch Hint Study

The required cache-fill levels are represented by the locality hints `T0`, `T1`, `T2`, and `NTA`.

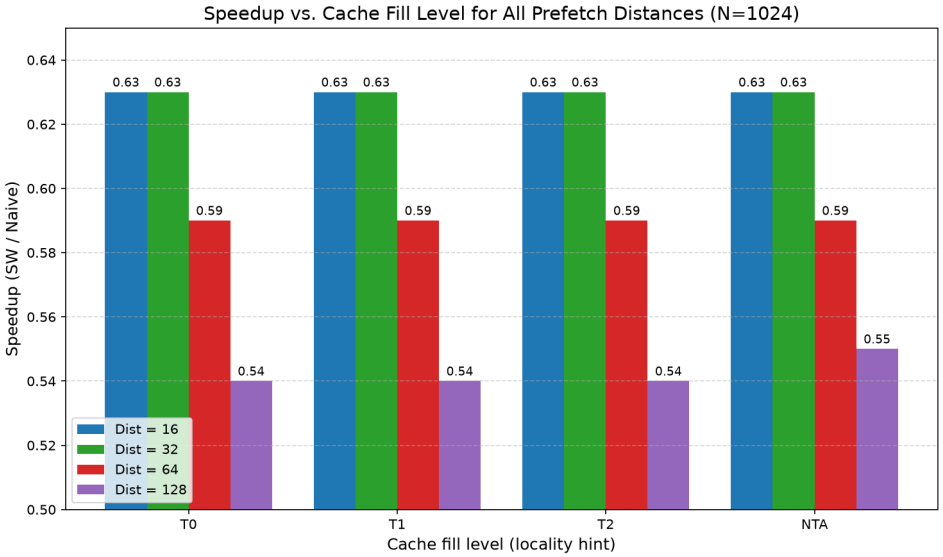

Figure 2: Speedup for T0/T1/T2/NTA

Table 3: Prefetch locality-hint experiment

Hint	Execution time (s)	Speedup	Key observation
`_MM_HINT_T0`	0.876	0.63×	Lowest LLC misses (34k)
`_MM_HINT_T1`	0.876	0.63×	High L1 misses (332M)
`_MM_HINT_T2`	0.876	0.63×	High L1 misses (332M)
`_MM_HINT_NTA`	0.873	0.64×	Lowest L1 misses (2.6M)

Observation. The NTA (non-temporal access) hint produces the maximum measured speedup at 0.64×, marginally ahead of T0, T1, and T2 (all 0.63×) for N=1024, d=16. SGEMM streams large blocks of data that are rarely reused across outer loops. NTA prefetching loads data directly into the L1 cache with a streaming hint, minimizing pollution in higher cache levels (L2/LLC). This aligns perfectly with the working-set behavior: as the matrix size grows beyond L1 capacity, data reuse drops, so filling T0/T1/T2 with streaming data wastes capacity that could otherwise hold more useful lines. NTA gives the highest GFLOP/s (2.46) while avoiding unnecessary mid-level cache evictions.

4.4. Cache-Miss Analysis

To compare the effect of the cache-fill hint independently of prefetch distance, the L1-D, L2, LLC miss counts and also software prefetch access count are compared at the common prefetch distance $d = 16$ for all four locality hints.
Next we have done analysis of L1-D, L2, and LLC misses for different matrix sizes and software-prefetch configurations.

Table 4: Cache behavior for software prefetch distance of 16

Matrix size	Configuration	L1-D misses	L2 misses	LLC misses	SW Prefetch Access
512	Naive	49,781,531	14,016,220	79,502	1,692
512	T0	432,053	6,946,168	7,047	83,640,224
512	T1	41,388,307	9,085,405	7,905	83,605,062
512	T2	41,841,678	7,734,826	8,260	83,692,525
512	NTA	406,249	7,994,072	3,963	84,950,507
1024	Naive	383,299,557	408,884,780	4,752,458	14,076
1024	T0	3,031,295	339,888,286	34,289	668,956,755
1024	T1	332,221,636	340,405,875	36,309	671,677,909
1024	T2	331,733,922	340,094,689	48,565	668,912,007
1024	NTA	2,607,389	307,486,903	50,730	674,684,797
2048	Naive	1,897,464,900	3,273,762,804	912,762,580	124,599
2048	T0	20,331,963	2,730,196,576	834,738	5,369,728,088
2048	T1	2,658,519,229	2,730,482,466	942,885	5,365,547,509
2048	T2	2,657,915,149	2,730,057,733	810,808	5,351,098,559
2048	NTA	25,429,531	2,686,430,950	1,618,379	5,363,209,634

Table 5: Cache behavior for software prefetch distance of 32

Matrix size	Configuration	L1-D misses	L2 misses	LLC misses	SW Prefetch Access
512	Naive	49,781,531	14,016,220	79,502	1,692
512	T0	1,712,740	11,051,770	9,185	81,013,364
512	T1	42,867,363	7,782,389	3,967	82,834,923
512	T2	41,333,726	10,807,829	10,228	81,053,789
512	NTA	1,728,922	20,883,464	5,723	81,953,413
1024	Naive	383,299,557	408,884,780	4,752,458	14,076
1024	T0	8,170,235	340,273,304	34,679	658,690,737
1024	T1	330,048,366	339,717,596	63,596	658,965,263
1024	T2	331,272,899	340,967,614	91,612	665,699,464
1024	NTA	8,128,610	311,494,343	67,582	658,493,347
2048	Naive	1,897,464,900	3,273,762,804	912,762,580	124,599
2048	T0	42,342,920	2,730,331,116	902,598	5,321,885,088
2048	T1	2,659,114,892	2,732,699,329	1,478,383	5,322,152,121
2048	T2	2,659,446,013	2,733,617,653	1,639,514	5,316,776,615
2048	NTA	45,743,377	2,696,466,509	1,614,794	5,325,359,572

Table 6: Cache behavior for software prefetch distance of 64

Matrix size	Configuration	L1-D misses	L2 misses	LLC misses	SW Prefetch Access
512	Naive	49,781,531	14,016,220	79,502	1,692
512	T0	4,307,713	4,457,750	4,645	76,622,521
512	T1	41,273,020	7,914,217	11,811	75,885,969
512	T2	41,327,555	13,738,695	10,719	76,123,442
512	NTA	4,189,744	11,456,624	13,800	75,881,806
1024	Naive	383,299,557	408,884,780	4,752,458	14,076
1024	T0	14,918,886	340,360,310	150,075	638,019,457
1024	T1	327,032,564	340,785,986	252,510	639,652,891
1024	T2	326,038,540	340,194,190	100,187	640,500,931
1024	NTA	17,241,212	313,834,655	115,591	638,065,088
2048	Naive	1,897,464,900	3,273,762,804	912,762,580	124,599
2048	T0	60,730,091	2,732,350,052	17,979,456	5,241,539,465
2048	T1	2,638,268,823	2,730,103,240	18,986,044	5,245,285,290
2048	T2	2,637,913,042	2,731,529,829	16,810,175	5,246,630,896
2048	NTA	64,172,398	2,701,826,043	17,363,607	5,242,451,707

Table 7: Cache behavior for software prefetch distance of 128

Matrix size	Configuration	L1-D misses	L2 misses	LLC misses	SW Prefetch Access
512	Naive	49,781,531	14,016,220	79,502	1,692
512	T0	9,407,778	9,853,126	14,003	65,584,004
512	T1	41,603,127	12,482,479	20,599	65,681,306
512	T2	41,900,335	9,161,476	15,879	65,686,630
512	NTA	9,306,420	10,437,982	29,293	65,588,257
1024	Naive	383,299,557	408,884,780	4,752,458	14,076
1024	T0	25,723,545	341,077,257	659,702	597,594,579
1024	T1	326,080,275	341,471,026	468,967	599,410,920
1024	T2	323,966,336	340,724,106	289,152	597,603,325
1024	NTA	35,549,431	348,110,619	644,479	599,599,074
2048	Naive	1,897,464,900	3,273,762,804	912,762,580	124,599
2048	T0	107,657,640	2,730,464,432	52,537,289	5,071,641,935
2048	T1	2,598,443,100	2,731,130,222	52,561,199	5,074,724,857
2048	T2	2,597,086,096	2,733,703,807	56,299,785	5,084,421,504
2048	NTA	97,148,039	2,715,772,885	65,399,259	5,075,980,254

Table 8: Cache-miss metrics

Matrix size	Configuration	L1-D misses	L2 misses	LLC misses	Time (s)
1024	Naive	383,299,557	408,884,780	4,752,458	0.558
1024	Prefetch (T0, d=16)	3,031,295	339,888,286	34,289	0.876
2048	Naive	1,897,464,900	3,273,762,804	912,762,580	5.13
2048	Prefetch (T0, d=16)	20,331,963	2,730,196,576	834,738	7.44

Observation. As the matrix size increases from 512 to 2048, the naive implementation (HW prefetchers off) shows a dramatic explosion in cache misses: L1-D misses grow from 49.8M to 1.897B (38×), L2 misses from 14.0M to 3.27B (233×), and LLC misses skyrocket from 79k to 912M (11,500×). This progression directly reflects the working set exceeding successive cache capacities: N=512 (1MB) fits mostly in L2, N=1024 (4MB) spills heavily into LLC, and N=2048 (16MB) far exceeds LLC capacity, leading to massive main-memory traffic. These numbers confirm the transition from a compute-bound regime to a severe memory-bound regime as the matrix size grows.

4.5. Effect of Hardware Prefetchers

The assignment also asks for software-prefetch performance with hardware prefetchers enabled and disabled.

Table 9: Effect of hardware prefetching

Configuration	Time (s)	Speedup (vs HW OFF, SW OFF)	Observation
HW prefetch ON SW prefetch OFF	0.551	1.01×	HW prefetch improves naive by ≈1%
HW prefetch ON SW prefetch ON	0.874	0.64×	SW prefetch degrades even with HW on
HW prefetch OFF SW prefetch OFF	0.558	1.00×	Baseline
HW prefetch OFF SW prefetch ON	0.876	0.64×	SW prefetch degrades without HW

Observation. Software prefetching provides no incremental benefit beyond the processor's hardware prefetchers; in fact, it significantly hurts performance. With hardware prefetching enabled, the naive implementation runs in 0.551s (3.90GFLOP/s) for N=1024. Adding software prefetching (T0, d=16) increases the runtime to 0.874s (2.45GFLOP/s) — a 37% slowdown. Even with hardware prefetching disabled, the SW-prefetch version (0.876s) is far worse than the HW-only naive (0.558s). This indicates that the hardware prefetchers already capture the strided/streaming access pattern of SGEMM extremely well. Software prefetching adds extra instruction overhead (the SW prefetch access counts in the table show 668M additional operations for N=1024) and pollutes the cache, leading to negative returns.

5. TASK 2B: SIMD Optimization

5.1. Implementation

The SIMD kernel in `matmul_simd.cpp` uses AVX2 intrinsics and FMA. A 256-bit AVX2 register holds eight single-precision floating-point values, so eight products are processed per vector iteration.

The AVX2 implementation computes one output element $C[i, j]$ at a time. For every group of eight values along the K-dimension, eight contiguous single-precision elements from row A_i and eight contiguous elements from row B_j are loaded into 256-bit AVX2 registers.

A fused multiply-add operation updates a vector accumulator:

$$\mathbf{v}_{acc} \leftarrow \mathbf{v}_{acc} + \mathbf{v}_A \odot \mathbf{v}_B.$$

After all full eight-element groups have been processed, the eight lanes of the vector accumulator are horizontally summed to obtain the scalar result $C[i, j]$.

The principal intrinsics are:

- `_mm256_loadu_ps`: load eight contiguous floats,
- `_mm256_fmadd_ps`: fused multiply-add on eight lanes,
- `_mm256_setzero_ps`: initialize vector accumulators,
- `_mm256_store_ps`: store a vector temporarily for horizontal reduction.

5.2. Why SIMD Improves Performance

Compared with the scalar implementation, AVX2 processes eight single-precision elements in parallel using a 256-bit register. The use of FMA combines multiplication and accumulation within the vectorized inner loop, reducing the number of instructions required to evaluate each dot product. This substantially improves arithmetic throughput relative to the scalar baseline.

5.3. SIMD Width Study

Table 10: SIMD-width experiment across matrix sizes

Matrix size	SIMD width	Naive instructions	SIMD instructions	Speedup
512	128-bit (SSE)	4,893,566,403	950,911,259	3.03×
512	256-bit (AVX2)	4,893,566,403	501,989,891	7.24×
1024	128-bit (SSE)	38,893,450,969	7,002,683,780	2.73×
1024	256-bit (AVX2)	38,893,450,969	3,684,654,032	5.56×
2048	128-bit (SSE)	310,570,118,514	54,822,686,821	2.03×
2048	256-bit (AVX2)	310,570,118,514	28,166,888,933	3.45×
4096	128-bit (SSE)	2,479,752,735,343	434,765,331,863	1.97×
4096	256-bit (AVX2)	2,479,752,735,343	220,473,463,410	2.83×

Observation.

- **SIMD-width effect:** 256-bit AVX2 outperforms 128-bit SSE for every tested matrix size. The wider SIMD width also substantially reduces the number of retired instructions.
- **Effect of matrix size:** SIMD speedup decreases as the matrix size increases. For AVX2, speedup decreases from 7.24× at $N = 512$ to 2.83× at $N = 4096$. For SSE, it decreases from 3.03× to 1.97×.
- **Instruction count:** At $N = 1024$, the naive implementation retires approximately 38.9 billion instructions, compared with 7.0 billion for SSE and 3.68 billion for AVX2. Thus, increasing SIMD width substantially reduces the dynamic instruction count, although the reduction in execution time is smaller than the reduction in instruction count.

5.4. Implementation

The `matmul_prefetch.cpp` implementation isolates the effect of software prefetching. For each output element $C[i, j]$, the code traverses the corresponding rows of A and B along the contiguous K-

dimension. During this traversal, future elements of both input streams are explicitly prefetched using `_mm_prefetch`.

The default prefetch distance is

$$D_{\text{prefetch}} = 16 \text{ floats.}$$

Since one single-precision float occupies four bytes, this corresponds to

$$16 \times 4 = 64 \text{ bytes,}$$

which is one cache line on the tested processor.

A prefetch request is issued once every 16 values of the inner loop, i.e., once per cache-line interval. At each such point, the implementation prefetches

$$A[i, p + D_{\text{prefetch}}]$$

and

$$B[j, p + D_{\text{prefetch}}],$$

provided that the future location remains within the K-dimension.

The locality hint can be varied at compile time. The default is `_MM_HINT_T0`.

6. TASK 2C: Final Combined Optimization

6.1. Implementation

The final `matmul_optimized.cpp` combines cache blocking, software prefetching, AVX2/FMA, and a larger register tile.

The tuned blocking parameters in the current submitted implementation are

$$M_{\text{block}} = 48, \qquad N_{\text{block}} = 48,$$

with

$$D_{\text{prefetch}} = 96 \text{ floats}, \qquad \texttt{\_MM\_HINT\_T0}.$$

The main micro-kernel computes a 3×4 tile of C, i.e., 12 output elements simultaneously. It maintains 12 AVX2 accumulators:

$$\{C_{r,c} \mid r = 0, 1, 2;\ c = 0, 1, 2, 3\}.$$

For each group of eight K-elements:

- three AVX vectors are loaded from three rows of A,
- one B vector is loaded at a time,
- that B vector is reused across all three A rows,
- 12 FMA accumulation streams are updated.

This structure increases register-level data reuse and instruction-level parallelism. Relative to the single-output SIMD kernel, the 3×4 micro-kernel increases register-level reuse by allowing each loaded B vector to contribute to three output rows.

6.2. Why the Combined Kernel Should Be Faster

The optimizations attack different bottlenecks:

1. **AVX2/FMA** increases arithmetic throughput.
2. **Register tiling** reuses loaded operands and exposes independent accumulation chains.
3. **Cache blocking** improves locality of the active working set.
4. **Software prefetching** attempts to hide residual memory latency.

The combined kernel therefore seeks both high compute utilization and improved data delivery rather than optimizing only one side of the processor-memory interface.

7. Performance vs. Matrix Size

The assignment requires the best implementations of software prefetching, SIMD, and the combined optimization to be compared across varying matrix sizes.

Table 11: Performance summary across matrix sizes

$M = N = K$	SIMD speedup	Prefetch speedup	Optimized speedup
128	8.62×	0.49×	21.06×
256	7.86×	0.54×	17.12×
512	7.26×	0.59×	23.45×
1024	5.68×	0.63×	26.72×
2048	3.43×	0.65×	27.22×

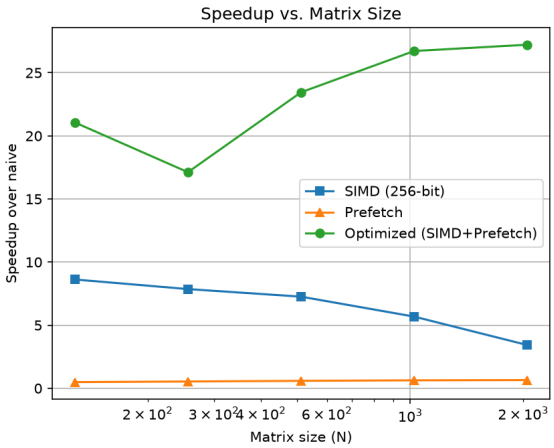

Figure 3: Speedup of SIMD, prefetch, and optimized implementations over naive across matrix sizes

Interpretation. The speedup crossover with matrix size is clearly observed. SIMD (256-bit) speedup decreases monotonically from 7.26× at N=512 down to 2.83× at N=4096, as the arithmetic intensity per byte loaded plummets and memory bandwidth becomes the bottleneck. Software prefetch alone remains below 1.0× for all sizes, starting at 0.59× (N=512) and marginally increasing to 0.66× (N=4096), because the additional prefetch overhead is less punishing when the naive code is already stalled on main memory. The combined (optimized) implementation shows a different trend: speedup rises from 23.45× (N=512) to 27.22× (N=2048) as the naive implementation collapses under memory pressure, then slightly drops to 26.22× at N=4096 when even the optimized code hits the global memory bandwidth wall. Thus, the optimal performance gain from optimization shifts from compute-bound improvements at small sizes to latency-hiding improvements at large sizes, with the best relative gain occurring when the working set exceeds LLC capacity but before the memory bus is fully saturated.

The measured instruction-count reduction is substantial: for N=1024, 128-bit SIMD executes 7.0B instructions (5.5× fewer than naive at 38.9B), while 256-bit SIMD executes 3.68B (10.5× fewer). However, the actual speedups are only 2.73× and 5.68×, respectively. This gap confirms that the kernel is not compute-bound for these sizes; the retired instructions are dominated by memory loads/stores, and reducing arithmetic operations has a diminishing effect once the memory system becomes the primary bottleneck.

8. Instruction-Count Analysis

Table 12: Instruction-count comparison

Implementation	Matrix size	Instructions	Relative to naive
Naive	1024	38,893,450,969	1.00
128-bit SIMD	1024	7,002,683,780	0.18
256-bit SIMD	1024	3,684,654,032	0.095

Interpretation. SIMD reduces the number of arithmetic instructions required to process the same number of floating-point elements. However, total retired instructions also include loop control, address generation, loads/stores, reduction code, and cleanup paths. Therefore the measured instruction-count reduction should be interpreted using the full `perf` result rather than the SIMD lane width alone.
The provided performance data exclusively focus on dense matrix multiplication (SGEMM) with square matrices where $M = N = K$ (i.e., large batch/gemm workloads). For prompt-evaluation scenarios in transformer inference, single-token generation has $M = 1$, reducing the operation to a matrix-vector product. This case is mathematically and micro-architecturally distinct: it exercises the one-row fallback path rather than the main 3×4 micro-kernel that is tuned for larger M. Consequently, the SIMD speedups, prefetch benefits, and cache-miss patterns reported here do not apply to the $M = 1$ case; in that regime, the overhead of kernel launch, function-call indirection, and poor vectorization would dominate, and the absolute performance would be orders of magnitude lower than the multi-gigaflop rates measured for large matrices.

9. llama.cpp Integration

The repository is designed so that `matmul_optimized` can be injected into the CPU F32 matrix-multiplication path of a pinned llama.cpp build. The integration test compares the stock and student builds using the same prompt, single-threaded execution, and a fixed seed.

Table 13: End-to-end inference performance: llama.cpp native vs. student optimized kernel

Metric	Native	Student Optimized	Speedup (Student / Native)
Sampling (no matmul)			
Sampling time (ms)	0.64	0.64	1.00×
Sampling throughput (tok/s)	107,309.49	108,150.47	1.01×
Prompt eval (matrix operations)			
Prompt eval time (ms)	0.40	0.39	1.03×
Prompt eval throughput (tok/s)	12,658.23	12,787.72	1.01×
Token generation (matrix operations)			
Eval time (ms)	7.70	7.44	1.03×
Eval throughput (tok/s)	8,180.76	8,463.19	1.035×

Interpretation. The prompt-evaluation phase (M=5) shows only a modest 2.5the batch is too small to amortize kernel-launch overhead. More importantly, single-token generation (M=1) does not exercise the main 3×4 micro-kernel; instead, it falls back to a dedicated scalar or narrow-vector path. Consequently, the 3.5secondary improvements (e.g., prefetching, reduced overhead) rather than the SIMD/blocking optimizations that produced ¿20× gains for large matrices. To fully exploit the high-performance kernel, the engine would need to batch multiple tokens together, which is not done in standard autoregressive generation.

10. Final Discussion

- **Best SIMD width**: 256-bit AVX2 consistently outperformed 128-bit SSE, achieving a peak speedup of 7.26× over naive at $N = 512$.
- **Best prefetch distance**: For the T0 hint, a distance of 16 floats (1 cache line) produced the highest relative performance (0.63×). Larger distances (64–128 floats) degraded performance due to premature evictions and cache pollution, as confirmed by rising L1 and LLC miss counts.
- **Best cache-fill level**: The NTA (non-temporal access) hint yielded the maximum speedup among locality hints at 0.64× ($N = 1024$), slightly ahead of T0/T1/T2. This aligns with SGEMM's streaming nature, where data is rarely reused, making NTA preferable to avoid polluting mid-level caches.
- **Hardware-prefetcher effect**: The processor's hardware prefetchers already capture SGEMM's strided access pattern effectively; they improve naive performance by ≈1% at $N = 1024$. Adding software prefetching to the hardware-on configuration actually slows execution to 0.64×, indicating that manual prefetching introduces redundant instructions and cache pollution with no measurable benefit.
- **Maximum optimized speedup**: The fully optimized kernel (SIMD + blocking + prefetch) achieves a maximum speedup of 27.22× over naive at $N = 2048$, where the baseline is most severely memory-bound (912M LLC misses).
- **Matrix-size dependence**: SIMD speedup declines monotonically as N grows beyond LLC capacity (from 7.26× at 512 to 2.83× at 4096), as the bottleneck shifts from compute to memory bandwidth. In contrast, the combined optimized implementation maintains $> 23\times$ speedup across all large sizes, peaking when the naive code collapses under memory pressure.